

Software Tools

Lecture 3: Regular Expressions

grep, sed, awk and everything they can do

Gretchen Hallett

Last Time...

We introduced shellscripting and showed how
to write more complex tools

We wrote a script together!

This time...

- We're going to go over regular expressions as a tool for matching stuff
 - We're going to introduce grep as a tool for searching
- We're going to introduce sed and awk as a way of processing text with regular expressions
- And I'm going to *mention* perl as a more general purpose way of doing all that

WHENEVER I LEARN A
NEW SKILL I CONCOCT
ELABORATE FANTASY
SCENARIOS WHERE IT
LETS ME SAVE THE DAY.

OH NO! THE KILLER
MUST HAVE FOLLOWED
HER ON VACATION!



BUT TO FIND THEM WE'D HAVE TO SEARCH
THROUGH 200 MB OF EMAILS LOOKING FOR
SOMETHING FORMATTED LIKE AN ADDRESS!



IT'S HOPELESS!

EVERYBODY STAND BACK.



I KNOW REGULAR
EXPRESSIONS.



IF YOU'RE HAVIN' PERL
PROBLEMS I FEEL
BAD FOR YOU, SON—



I GOT 99
PROBLEMS,



SO I USED
REGULAR
EXPRESSIONS.



NOW I HAVE
100 PROBLEMS.



Regular expressions are a *language* for matching text

- Based on a *finite state* machine
- *Notoriously* finicky and brittle
- *Incredibly useful* for 99% of text based problems, in practice

Also called *re*, *regex*, *regexp*...

A perfectly normal Regular expression to match email addresses (that *almost* works)

```
(?:[a-z0-9!#$%&'*/=?^_`{|}~-]+(?:\. [a-z0-9!#$%&'*/=?^_`{|}~-]+)*|"(?:  
    [\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-  
    \x09\x0b\x0c\x0e-\x7f])*")@(?:(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.|  
    [a-z0-9](?:[a-z0-9-]*[a-z0-9])?|\[(?:(?:(25[0-5]|2[0-4][0-9]|  
    [01]?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|  
    [01]?[0-9][0-9]?)|[a-z0-9-]*[a-z0-9]:  
    (\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f)|  
    \[ \x01-\x09\x0b\x0c\x0e-  
    \x7f])+) \))
```

WTF is that piece of...?!

Regular expressions have a habit of getting out of hand quickly

Consider them *write-only* code

- ...Because you will forget how the work

They are a *regular language* so are bounded in complexity... (*no arbitrary recursion*)

Their syntax is easy though...

Regular expressions we're first described back in the 1950s by Stephen Cole Kleene...

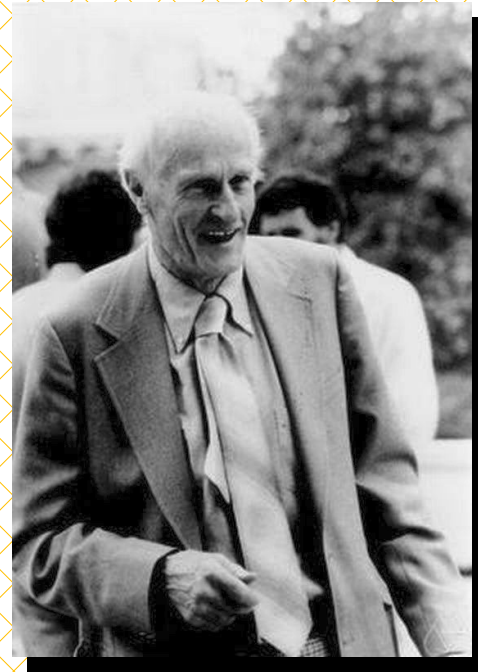
They were widely integrated in UNIX tools back before there were standards...

By the time POSIX standardized things there were multiple competing standards

- POSIX: Basic and Extended RE
- Perl and Perl Compatible Regular Expressions (PCRE)

We will stick to POSIX Basic RE...

- But I use Perl in real life



How to read a regular expression

Start at the left one item at a time...

- If it matches the string then keep going
- Otherwise backtrack and try again (possible at the next character)

Most regular expressions are *greedy*

- They will always match *as much as possible*

Matches any character

^

Matches the start of a line

\$

Matches the end of a line
(don't ask about multiline regexp)

brian.

**Matches a 'b' followed by an
'r' followed by an 'i' followed
by an 'a' followed by an 'n'
followed by anything**

[nigel]

**Matches one of any of the characters in
the word 'nigel'**

[^nigel]

**Matches anything but one of the
characters in the word 'nigel'**

[a-z0-9]

Matches any character in the space 'a' through to 'z' and '0' through to '9'

\(\)

Matches whatever is inside the \(\)
AS A SINGLE STANDALONE THING

**why does the
standalone
bit matter\?**

**Matches “why does the standalone bit
matte” or “why does the standalone bit
matter”**

\?

**Matches the thing before it once or zero
times**

colou\?r

**matches English and American spellings
of “colour”**

\+

**Matches the thing that came before it
one or more times**

\+ \?

**Matches the thing that came before it
one or more times, zero or one time**

**Matches the thing that came before it
zero or more times
(no I'm not changing color for this)**

squ\ (ee\)\+

**Matches excited noises with an even
number of 'e's**

squ\(e\{2\}**\)+**

**Matches excited noises with an even
number of 'e's**

squ\(e\{2,3\}**\)+**

**Matches excited noises with a number of
'e's that's a multiple of two or three
(leaving numbers off either side of the comma makes it
an open range)
(ooh we're back to red!)**

\1

**Matches whatever was matched within
the nth (normally 1-9) \[\] expression**

\([[:alpha:]]+\) \1 \1

**Matches *triple tautonyms* (animals
whose Latin name is the same word three
times like 'gorilla gorilla gorilla')**

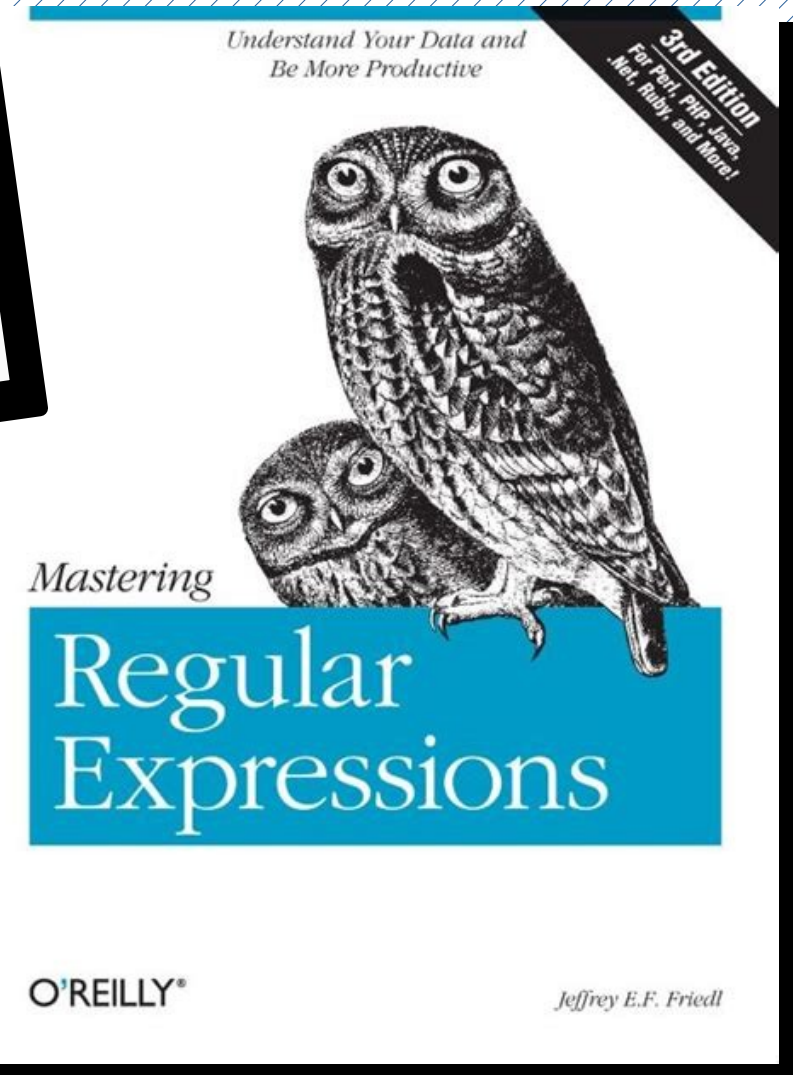
Congratulations!

You now know regular expressions

(The extended and Perl ones usually leave off the backslashes and make you escape the literal characters; and in the Perl case add a TONNE useful extra stuff)

If you want to know
more I like this
book (but it's rather
old now...)

By more I mean how
they're implemented and
don't just wanna read a
load of theory papers (but
you *should* read the theory
papers they're great)



So I've given you
this **great** tool
for matching text
how are we going
to use it?

grep for search

sed for replace
(and more)

awk for more general
manipulation

perl as a
programming language

grep is a search tool for text documents.

It's name comes from the ed editor
command to search

global /regexp/ print → g/r e/p

It prints all lines in its input that match a
regexp passed as an argument

Faster alternatives
exist... (see ripgrep)
but don't behave the
same, or aren't always
available

-i :: case insensitive

-E or -P :: extended or Perl (non-POSIX) regexp

-o :: only matching string not line

-v :: inverted matching

-R :: search folder recursively

How many words in
English contain
double letters?

I have a dictionary at /usr/share/dict/words...
I can remember the wc command can count lines...

```
$ grep -i '\(.\)\'1' /usr/share/dict/words | wc -l  
50988
```

```
# What about 2 or more double letters? (-c for count)  
$ grep -ic '\(.\)\'1.*\(.\)\'2' /usr/share/dict/words  
4137
```

```
# Lets simplify...  
$ grep -ic '\(\(.\)\'2.*\)\{2\}' /usr/share/dict/words  
4137
```

```
# What about 3 or more?  
$ grep -ic '\(\(.\)\'2.*\)\{3\}' /usr/share/dict/words  
170
```

```
# So where does it end?
# seq generates a sequence of numbers (man seq)...
# column formats tabluar data
$ for n in $(seq 2 6); do
    hits=$(grep -ic "\\(\\(\\.\\)\\2.*\\)\\{\\$n\\}" words)
    printf "%d double letters,%d\n" "$n" "$hits"
done | column -ts ,
```

```
2 double letters:  4137
3 double letters:  170
4 double letters:   4
5 double letters:   0
6 double letters:   0
```

```
# (Yes I hate escaping regex in strings... lookup Perl ones)
# killeekillee possessionlessness subbookkeeper successlessness
# Wow, possessionlessness it's always 'ss',
# ...and 'subbookkeeper' it's always a different letter and...
# ...they're one after another! That's really cool!
```

sed is a programmable
editor for streams (pipes
and files)

- n :: don't print lines by default
- E :: extended regexp
- i :: edit files in place

The command syntax
says has is borderline
unintelligible, especially
if you want to write long
complicated things with
lots of branching.

It basically looks like a
kind of horrific single
letter assembly
language. For any *real*
problem I'd use a
proper programming
language, but for *quick*
jobs it is indispensable

```
# Look you can grep with sed!  
$ sed -n '/\(\(.\)2.*\) \{4\}/p' /usr/share/dict/words  
killeekillee  
possessionlessness  
subbookkeeper  
successlessness
```

```
# The // bit matches the line, and the p prints it  
# Lets make the double letters upper case...  
$ </usr/share/dict/words sed -n '/\(\(.\)2.*\) \{4\}/p' |  
  gsed 's/\(.\)1/\U11/g'  
kiLLEEkilLEE  
poSSeSSionleSSneSS  
suBB00KKEEper  
suCCeSSleSSneSS
```

```
# nb. POSIX sed doesn't have the \U uppercase trick  
# I've used GNU sed (the default on Linux) instead here  
# which is installed on my system as gsed
```

The sed s command works as follows

s/target/replacement/flags

Where target and replacement are regular expressions

The most common flags you'll encounter are:

- g :: do the replacement more than once per-line
- i :: do the replacement case insensitively

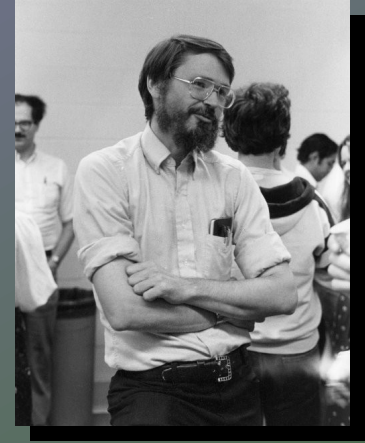
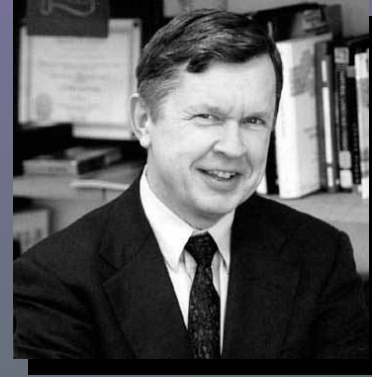
There is also a y command that does substitution ciphers between ranges of letters

```
$ echo "0x1FFF3BB0" | sed 'y/ABCDEF/abcdef/'
```

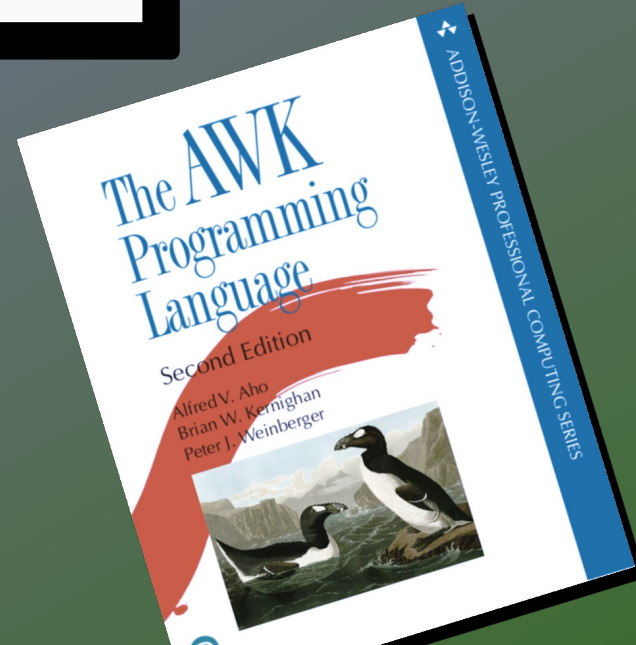
...but I'd use the tr command instead of sed here

99% of what you'll use sed for is search and replace

awk is for when you need to deal with semi structured data. It is a programming language with a focus on line based input and manipulation



Named for:
Alfred Aho
Peter Weinberger
Brian Kernighan



Me and Matt have gone
for lunch!

We have our receipt that
looks like the following:

Orderer

Item ordered, Cost

\$ cat receipt

Gretch

Salad, 4.99

Iced Tea, 2.99

Cake, 2.00

Matt

Steak, 19.99

Whiskey (Neat), 10.00

Another steak, 19.99

```
# Lets leave the money out of it
# To do this lets set our field separator to , and just print
# the first thing
```

```
# In AWK everything in the { } block is run on every line...
```

```
$ awk -F, '{print $1}' <receipt
```

```
# Gretch
```

```
Salad
```

```
Iced Tea
```

```
Cake
```

```
# Matt
```

```
Steak
```

```
Whiskey (Neat)
```

```
Another steak
```

No we need money,
Matt'll pay he's a
gentleman

but he needs a total...

BEGIN { } and END { }
blocks are run at the
beginning or the end.

Note that some things
are implicit: print on
its own prints the
whole line (like print
\$0); I didn't
initialize sum either
(BEGIN { sum=0 })

```
$ awk -F, '
{
    sum+=$2;
    print
}
END {
    print "\n# Total\n-----," sum
}' <receipt
! column -ts,
```

```
# Gretch
Salad                4.99
Iced Tea             2.99
Cake                 2.00
# Matt
Steak                19.99
Whiskey (Neat)      10.00
Another steak       19.99
# Total
-----
59.96
```

```

$ awk -F, '
    {sum+=$2;
    print $0, " ", sum
    } END {
    print "\n# Total\n-----," sum
    }' <receipt | column -ts,

```

# Gretch	0	
Salad	4.99	4.99
Iced Tea	2.99	7.98
Cake	2.00	9.98
	9.98	

# Matt	9.98	
Steak	19.99	29.97
Whiskey (Neat)	10.00	39.97
Another steak	19.99	59.96
# Total		
-----	59.96	

No a running total...

Urgh it's impacting
on the named headers...

If we add /re/ before a
block we can choose
whether to apply it

\$2 ~ /re/ tests whether
field 2 matches the
regex

```
$ awk -F, '
    $2~/^$/ { print }
    $2!~/^$/ {
        sum+=$2;
        print $0, " ",sum;
    } END {
        print "\n# Total\n-----" sum
    }' <receipt | column -ts,
# Gretch
Salad                4.99      4.99
Iced Tea             2.99      7.98
Cake                 2.00      9.98
# Matt
Steak                19.99     29.97
Whiskey (Neat)       10.00     39.97
Another steak        19.99     59.96
# Total
-----
                    59.96
```

No I should really
pay my own bills, even
if Matt does want to be
nice... and besides he
spent 5x what I did...

See this is the stuff
awk is good at... you
can do what you could
easily do in a
spreadsheet on the
commandline

*(In roughly the same
amount of time if you
spend years learning
awk... I'm not selling
this well but still)*

```
$ awk -F, '($2~/^$/ {
    if(sum>0){
        print "-----," sum;
        sum=0;
    }
    print;
}' <receipt | column -ts,

# Gretch
Salad                4.99      4.99
Iced Tea             2.99      7.98
Cake                 2.00      9.98
-----
                       9.98

# Matt
Steak                19.99     19.99
Whiskey (Neat)      10.00     29.99
Another steak       19.99     49.98
-----
                       49.98
```

There is *plenty* more you can do
with awk... but I wanna move on...

Tools like the shell, grep, sed and
awk exist as separate utilities because
they were written in anger by people
needed a quick tool to do a job...

UNIX philosophy dictates *write small tools that do **one thing well**...*

But when programming we don't want to have to keep the syntaxes of 12 different tools in our heads

Perl is a general purpose programming language, that has a ***lot*** of features and workflow around manipulating text as we've been doing...

It's basically a more stable, and more portable version of shellscript with a lot of nice tools baked in...



Raku (*formerly Perl 6*) is a fork of the language with more cool things (*PEG grammars!*) that's slightly more modern (but still very much Perl)



https://theweeklychallenge.org

● CHALLENGE ACTIVE

The Weekly Challenge

“

”

You **don't** have to be *an expert*



TASK #1

Alternate Case

You are given a string containing an equal number of uppercase and lowercase English letters. Write a script to the minimum number of adjacent character swaps needed to turn the given string into an alternate case string.

TASK #2

Alternating Vowels Consonants

You are given three strings containing English alphabetic characters. Find all the longest contiguous substrings common to all three strings that strictly alternate between vowels and consonants.

🔍 TAKE THE CHALLENGE

👉 HOW TO CONTRIBUTE

🏆 SPONSOR US

👥 GUEST CONTRIBUTIONS

🏆 LATEST CHAMPION

🐪 PERL REVIEW

🦋 RAKU REVIEW

📺 YOUTUBE

```
BEFOREHAND: close door, each window & exit; wait until time.  
    open spellbook, study, select it, confess, tell, deny;  
write it, print the hex while each watches,  
    reverse "its length", write again;  
    kill spiders, pop them, chop, split, kill them.  
        unlink arms, shift, wait & listen (listening, wait),  
sort the flock (then, warn "the goats". kill "the sheep");  
    kill them, dump qualms, shift moralities,  
    values aside, each one;  
        die sheep, die, reverse system  
        you accept (reject, respect);  
next step,  
    kill next sacrifice, each sacrifice,  
    wait, redo ritual until "all the spirits are pleased";  
    do it ("as they say").  
do it(*everyone***must***participate***in***forbidden**s*e*x*).  
return last victim; package body;  
    exit crypt (time, times & "half a time") & close it,  
    select (quickly) & warn next victim;  
AFTERWORDS: tell nobody.  
    wait, wait until time;  
    wait until next year, next decade;  
        sleep, sleep, die yourself and  
        rest at last
```